Lab 5

x86-64 Assembly: Stack Overflow

Buffer Slide

Discussing doubts/solutions attempts for last week's inlab / labexam / outlab

Outlabs 3 & 4 are out. Inlab 5 is as well.

All previous labexam solutions are uploaded on the repo

Plan for this lab

- ➔ How to handle **decimal** numbers and **Floating point** operations
- ➔ Breaking code into **functions** and understanding the **stack**
- ➔ **Syscalls :** Use some predefined functions which can be used to call the OS

XMM Registers

These are special 128 bit registers used for floating point arithmetic (wired up to the parts of the cpu which have capabilities of performing floating point arth)

- **`xmm0-xmm15`** are the 16 xmm registers present in 64 bit systems. (In 32 bit systems only 8 xmm registers are present)
- **`SSE Instruction`** are special instructions that are present in the ISA to perform operations with these registers.
- **`So how do we use them ?`**

```
add_cmplx:

    movsd xmm0, [rdi+8]
    movsd xmm1, [rdi+16]
    addsd xmm0, [rsi+8]
    addsd xmm1, [rdi+16]
    movsd [rdx+8], xmm0
    movsd [rdx+16], xmm1
    ret
```

Basic xmm/SSE instructions

`movsd xmm, xmm/*mem` `movsd xmm/*mem, xmm`	Can move data to / from memory location or other xmm
`addsd xmm, xmm/*mem` `subsd xmm, xmm/*mem`	add / subtract, in place
`mulsd xmm, xmm/*mem` `divsd xmm, xmm/*mem`	multiply / divide in place
`movq xmm, reg/*mem` `movq reg/*mem, xmm`	Can move data to / from general purpose register

We are only using **`64 bits`** out of the **`128 bits`** width in xmm registers

It’s inlab (1) time

Best Functions Forever <3

- **`call <label>:`** Pushes the address of the **next** instruction (also known as the return address) and jumps to the label.
- **`ret:`** Pops the return address from top of the stack and jumps back to respective address

```
main:
    call my_func
    ; execution resumes here

my_func:
    ; do work
    ret
```

Careful what you wish for

- What if we push and pop something else on the stack before doing ret?
- ret will read **something else** and it will not return :(

What is a stack?

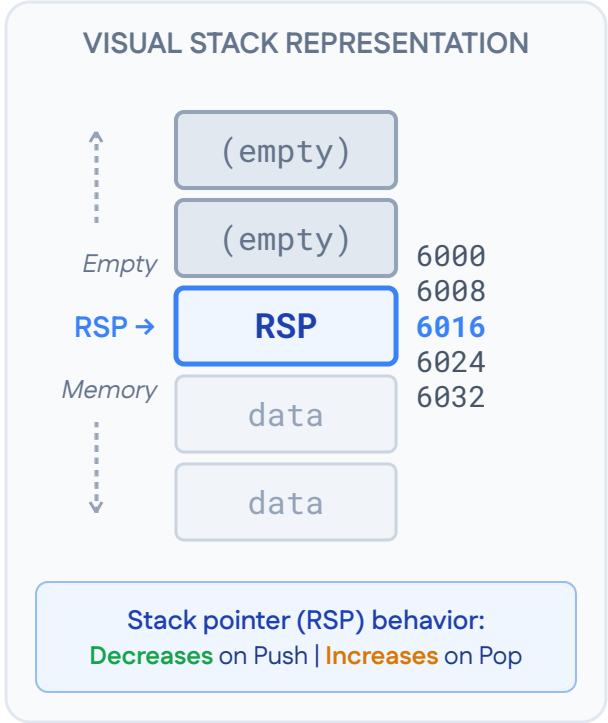
- Recall the function stack mumbo jumbo in c++
- We push things on the stack before a function call

  And pop them afterwards
- Stack grows downwards! (Higher -> lower memory addresses)
- Push operation: eg: `push rax`
- Equivalent assembly:

```
sub rsp, 8       ; say initially rsp=6016
mov [rsp], rax
```

- Pop operation: eg `pop rax`
- What should be equivalent assembly?

```
mov rax, [rsp]
add rsp, 8
```



Caller Callee Conventions

- **Caller saved registers:**
  - **Function arguments:** `rdi, rsi, rdx, rcx,r8, r9`
  - **Return value:** `rax`
  - **Scratch:** `r10, r11`
  - **All xmm registers:** `xmm0-xmm15`
- **Callee saved registers:**
  - **General Purpose Registers:** `rbx, rsp, rbp, r12, r13, r14, r15`

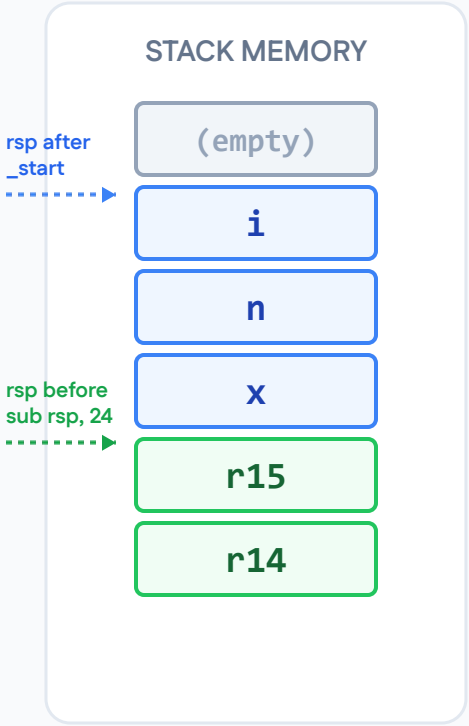
Example: Function

```
_start:
    push rbp
    push rsp
    push rbx
    push r12
    push r13
    push r14
    push r15

    sub rsp, 24

    mov rax, 0
    mov [rsp + 16], rax ; x = 0
    mov rax, 3
    mov [rsp + 8], rax ; n = 3

    mov rax, 0
    mov [rsp], rax ; i = 0
```



main.c

```
// Equivalent in C
long long x = 0;
long long n = 0;
long long i = 0;
```

Alignment is key

```
void example() {
    char a = 'A';  // 1 byte  (offset -1)
                   // 3 bytes of PADDING added here
    int b = 42;    // 4 bytes (offset -8, must be 4-byte aligned)
    char c = 'C';  // 1 byte  (offset -9)
                   // 7 bytes of PADDING added here
    double d = 3.14; // 8 bytes (offset -24, must be 8-byte aligned)
}
```

- Recall memory alignment of data types from the theory course
- TLDR: **`sizeof(T) % alignof(T) == 0`**
- This is a standard practice (called SysV ABI) that C/C++ compilers implement for us
- In x86 we will have to manually insert “padding” to achieve this
- **Padding** – leaving some gaps in the memory address space

Alignment is key

```
void example() {
    char a = 'A';  // 1 byte  (offset -1)
                   // 3 bytes of PADDING added here
    int b = 42;    // 4 bytes (offset -8, must be 4-byte aligned)
    char c = 'C';  // 1 byte  (offset -9)
                   // 7 bytes of PADDING added here
    double d = 3.14; // 8 bytes (offset -24, must be 8-byte aligned)
}
```

- For eg: for `int b` -> starting address should be ≡0 mod 4
- We insert padding suitable for this
- Remember xmm? Address should be 8 byte aligned while using `movsd` and `movaps/movapd` etc. Note: `movups` allows unaligned!

- When you call a function make sure its **16 byte aligned (at `call` boundary)**
- Especially when you use C functions like `printf` etc.. Why?

General functions and their convention

Register Arguments (Passed in Order)

1st Arg	2nd Arg	3rd Arg	4th Arg	5th Arg	6th Arg
`rdi`	`rsi`	`rdx`	`rcx`	`r8`	`r9`

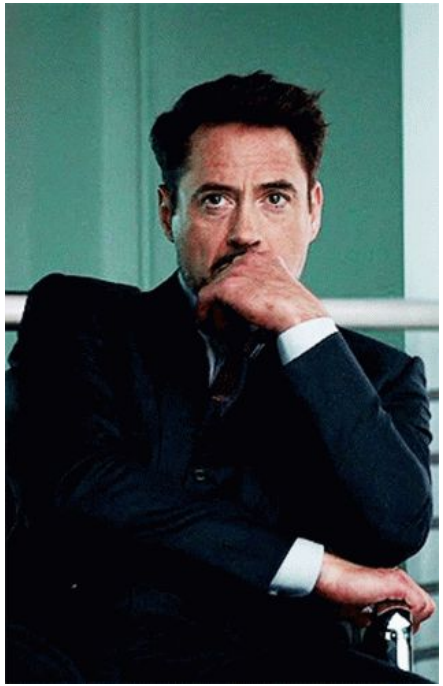
RETURN VALUE REGISTER

`rax`

Random questions

Common Doubts

- Why follow caller-callee convention?
- Why push/pop registers before & after every call?
- Why follow argument and return value conventions at all?



The Answer

MODULARITY

- Allows seamless integration with **C function libraries**.
- Enables using functions written by other people.
- Required to perform standard **system calls**.
- Mutual convention ensures code interoperability.

How do we use **C functions** in assembly?? (CHECK OUT OUTLAB)

Syscall me on my cell phone

What is a Syscall?

Requesting a privileged action from the OS (like writing to stdout).

The Mechanism

1. Load the syscall ID into `rax` (e.g., 1 for sys_write).
2. Load arguments using standard ABI rules (`rdi`, `rsi`, `rdx`).
3. Substitute `r10` in place of `rcx` if used
4. Execute the **`syscall`** instruction.

Position · Register · Description (sys_write Example)

Syscall Number `rax` (1 for `sys_write`)

1st Argument `rdi` (File Descriptor (`1` for `stdout`) for write)

2nd Argument `rsi` (Pointer to buffer string (`msg`) for write)

3rd Argument `rdx` (Length of buffer (`msglen`) for write)

**Note: Register arguments (rdi, rsi, rdx, etc.) hold entirely different meanings depending on the specific syscall invoked.*

It’s inlab (2 & 3) time

Caller callee convention done well

- Following are some slides to give a better understanding of caller callee convention
- Optional resources:
  - Calling convention (wiki)
  - A good outline of calling convention
  - (manual) the full sysv abi which goes way beyond just caller-callee register convention

- In short, registers are limited and common for any piece of code running on the CPU. This is a set of conventions for where and how to save each register.

Good old caller-callee

- A simple example of good old caller callee convention is on the repo here

CALLER – precall and postcall work

```
_start:
    ; GOAL: compute adder(10, 20) * 3 - 10

    mov rdi, 10 ; rdi - caller saved
    mov rbx, 3  ; rbx - callee saved

    ; PRE-CALL WORK : push caller-saved regs - rax,rdi,rsi,rdx,rcx,r8-11,xmm*
    ; Q1: Why am I not pushing rax? See below at NOTE1
    push rdi
    push rsi
    push rdx
    ; ...

    mov rdi, 10   ; put arguments in rdi, rsi...
    mov rsi, 20
    call adder    ; answer is put in rax

    ; POST-CALL WORK: pop caller-saved regs
    pop rdx
    pop rsi
    pop rdi
    ; recover old rdi, rsi, rdx values

    ; NOTE1: Does this usage give you the answer to Q1?
    imul rax, rbx ; rax *= rbx (rbx is untouched by `adder`, the callee)
    sub rax, rdi  ; rax -= rdi (rdi is pushed/popped by me , the caller)
    mov r12, rax  ; stash it away in r12, r12 is my returncode value

    ; answer (came in `rax`) should sit in `rdi`
    ; for setting returncode in syscall
    mov rdi, r12
    mov rax, 60
    syscall
```

CALLEE – prologue and epilogue

```
; receives arguments in rdi, rsi, rdx, rcx, ...
adder:
    ; PROLOGUE: push callee-saved regs - rbx,rsp,rbp,r12-15
    push rbx
    push r12

    ; do work!!
    ; here i can tamper with rbx
    mov rbx, rdi
    add rbx, rsi
    mov rax, rbx ; finally store return value in rax

    ; EPILOGUE: pop callee-saved regs - rbx,rsp,rbp,r12-15
    pop r12
    pop rbx

    ret
```

An Example with Godbolt

Simple C++ code and its assembly using the caller callee convention. [Godbolt link]

The code is on the repo as `sum.c` and `sum.s`. (Assembly is slightly modified for teaching)

- C++ code. Remember every variable is a register.
- Pause here and think which registers you should use for each variable – **caller-saved** or **callee-saved**. What are the tradeoffs?

```
1   float scale_add(float a, float b) {       // leaf
2       return a * 3.0f + b;
3   }
4
5   int sum_loop(float *arr, int n) {     // caller: has a value (total) live ACROSS the ca
6       float total = 0;
7       for (int i = 0; i < n; i++) {
8           total += scale_add(arr[i], i);   // total must survive this call
9       }
10      return total;
11  }
```

An Example with Godbolt

- Simple C++ code and its assembly using the caller callee convention. [Godbolt link]
- Callee (the leaf function code)
  - Do I really need to use `rbx`? Now imagine a world with callee-saved-only convention!

    Remember that this function is called in a **loop** multiple times

```
; ---- LEAF CALLEE: borrows rbx as scratch -> must save/restore it ----
scale_add(float, float):
        push    rbx             ; PROLOGUE: push rbx on stack
        mov     ebx, 3          ; ebx = 3   (just using rbx as scratch space)
        cvtsi2ss xmm2, ebx      ; xmm2 = 3.0
        mulss   xmm0, xmm2      ; xmm0 = a * 3.0
        addss   xmm0, xmm1      ; xmm0 += b     -> return value in xmm0
        pop     rbx             ; EPILOGUE: restore rbx
        ret
```

An Example with Godbolt

- Caller

```
; ---- CALLER ----
sum_loop(float*, int):
        xorps   xmm0, xmm0 ; total = 0.0, kept directly in xmm0
        mov     r12d, 0    ; i = 0
.loop_check:
        cmp     r12d, esi             ; i vs n (n is chilling in rsi)
        jge     .loop_end
        push    rdi                   ; PRE-CALL WORK: save 'arr'
        push    rsi                   ; PRE-CALL WORK: save 'n'
        push    r12                   ; PRE-CALL WORK: save 'i'
        sub     rsp, 8                ; PRE-CALL WORK: make room on stack for 'total'
        movss   dword ptr [rsp], xmm0 ; PRE-CALL WORK: save 'total'

        ; setup call
        movss   xmm0, dword ptr [rdi + 4*r12] ; xmm0 = arr[i]   (arg 1 for scale_add)
        cvtsi2ss xmm1, r12d                   ; xmm1 = (float)i (arg 2 for scale_add)
        call    scale_add(float, float)

        addss   xmm0, dword ptr [rsp] ; POST-CALL WORK: total += return value
        add     rsp, 8                ; POST-CALL WORK: free stack slot used for 'total'
        pop     r12                   ; POST-CALL WORK: restore 'i'
        pop     rsi                   ; POST-CALL WORK: restore 'n'
        pop     rdi                   ; POST-CALL WORK: restore 'arr'
        inc     r12d                  ; i++
        jmp     .loop_check
.loop_end:
        ret          ; total is already sitting in xmm0, nothing more to do
```